Layr Labs

Sidecar Rewards v2.2

Round 2 Security Assessment Report

Version: 2.0

February, 2026

Contents

Introduction 2
- Disclaimer . . . 2
- Document Structure . . . 2
- Overview . . . 2

Security Assessment Summary 3
- Scope . . . 3
- Approach . . . 3
- Coverage Limitations . . . 3
- Findings Summary . . . 3

Detailed Findings 5

Summary of Findings 6
- Missing Allocation Factor In Unique-Stake Staker Weighting . . . 7
- Silent Failure In Staker Delegation Snapshot Generation . . . 9
- Rewards-By-AVS Pagination Offset Uses Page Number Directly . . . 10
- Owner Split Misconfiguration Can Create Negative Staker Reward Math . . . 11
- Missing Strategy Registration Enforcement In V2.2 Stake Rewards Path . . . 13
- Token Filter SQL Uses Inconsistent Placeholders . . . 15

A Vulnerability Severity Classification 17

Introduction

Sigma Prime was commercially engaged to perform a time-boxed security review of the Layr Labs components in scope. The review focused solely on the security aspects of these components, though general recommendations and informational comments are also provided.

Disclaimer

Sigma Prime makes all effort but holds no responsibility for the findings of this security review. Sigma Prime does not provide any guarantees relating to the function of the components in scope. Sigma Prime makes no judgements on, or provides any security review, regarding the underlying business model or the individuals involved in the project.

Document Structure

The first section provides an overview of the functionality of the Layr Labs components contained within the scope of the security review. A summary followed by a detailed review of the discovered vulnerabilities is then given which assigns each vulnerability a severity rating (see Vulnerability Severity Classification), an *open/closed/resolved* status and a recommendation. Additionally, findings which do not have direct security implications (but are potentially of interest) are marked as *informational*.

The appendix provides additional documentation, including the severity matrix used to classify vulnerabilities within the Layr Labs components in scope.

Overview

Sidecar is an off-chain service in the EigenLayer stack that indexes on-chain state and events, builds reward-related snapshots, and computes reward distributions across stakers, operators, and operator sets. It also provides query surfaces used by downstream services to retrieve reward and accounting data derived from those computations.

This assessment is the second review round and focuses on the Sidecar Rewards v2.2 changes in scope, with emphasis on correctness and safety of reward calculation and snapshot-generation logic.

Security Assessment Summary

Scope

The review was conducted on the files hosted on the Layr-Labs/sidecar repository.

The scope of this time-boxed review was strictly limited to files at pull 500.

Note: third party libraries and dependencies were excluded from the scope of this assessment.

Approach

The security assessment covered components written in Golang.

For the Golang components, the manual review focused on identifying issues associated with the business logic implementation of the libraries and modules. This includes their internal interactions, intended functionality and correct implementation with respect to the underlying functionality of the Go runtime.

Additionally, the manual review process focused on identifying vulnerabilities related to known Golang anti-patterns and attack vectors, such as integer overflow, floating point underflow, deadlocking, race conditions, memory and CPU exhaustion attacks, and various panic scenarios including `nil` pointer dereferences, index out of bounds, and explicit panic calls.

To support the Golang components of the review, the testing team may use the following automated testing tools:

- golangci-lint: `https://golangci-lint.run/`
- vet: `https://pkg.go.dev/cmd/vet`
- errcheck: `https://github.com/kisielk/errcheck`

Output for these automated tools is available upon request.

Coverage Limitations

Due to the time-boxed nature of this review, all documented vulnerabilities reflect best effort within the allotted, limited engagement time. As such, Sigma Prime recommends to further investigate areas of the code, and any related functionality, where majority of critical and high risk vulnerabilities were identified.

Findings Summary

The testing team identified a total of 6 issues during this assessment. Categorised by their severity:

- High: 1 issue.
- Medium: 1 issue.

- Low: 3 issues.
- Informational: 1 issue.

Detailed Findings

This section provides a detailed description of the vulnerabilities identified within the Layr Labs components in scope. Each vulnerability has a severity classification which is determined from the likelihood and impact of each issue by the matrix given in the Appendix: Vulnerability Severity Classification.

A number of additional properties of the components, including optimisations, are also described in this section and are labelled as "informational".

Each vulnerability is also assigned a **status**:

- ***Open:*** the issue has not been addressed by the project team.
- ***Resolved:*** the issue was acknowledged by the project team and updates to the affected components(s) have been made to mitigate the related risk.
- ***Closed:*** the issue was acknowledged by the project team but no further actions have been taken.

Summary of Findings

ID	Description	Severity	Status
ELSD-01	Missing Allocation Factor In Unique-Stake Staker Weighting	High	Resolved
ELSD-02	Silent Failure In Staker Delegation Snapshot Generation	Medium	Resolved
ELSD-03	Rewards-By-AVS Pagination Offset Uses Page Number Directly	Low	Resolved
ELSD-04	Owner Split Misconfiguration Can Create Negative Staker Reward Math	Low	Resolved
ELSD-05	Missing Strategy Registration Enforcement In V2.2 Stake Rewards Path	Low	Closed
ELSD-06	Token Filter SQL Uses Inconsistent Placeholders	Informational	Resolved

ELSD-01	Missing Allocation Factor In Unique-Stake Staker Weighting		
Assets	`sidecar/pkg/rewards/16_goldOperatorOperatorSetUniqueStakeRewards.go` `sidecar/pkg/rewards/17_goldStakerOperatorSetUniqueStakeRewards.go`		
Status	**Resolved:** See Resolution		
Rating	Severity: High	Impact: Medium	Likelihood: High

Description

In the unique-stake reward path, operator-level weighting applies the allocation factor `magnitude / max_magnitude`, but staker-level weighting does not. This creates an inconsistency when an operator has different allocation ratios across strategies in the same operator set.

In `16_goldOperatorOperatorSetUniqueStakeRewards.go`, operator weight is allocation-adjusted before pro-rata distribution:

```
-- Step 2: Calculate allocated weight per operator
operators_with_weight AS (
    SELECT
        rso.*,
        SUM(
            oss.shares *
            oas.magnitude /
            oas.max_magnitude * rso.multiplier
        ) OVER (PARTITION BY rso.reward_hash, rso.snapshot, rso.operator) as operator_allocated_weight
    FROM registered_operators rso
    JOIN {{.operatorAllocationSnapshotsTable}} oas
        ON rso.operator = oas.operator
        AND rso.avs = oas.avs
        AND rso.strategy = oas.strategy
        AND rso.operator_set_id = oas.operator_set_id
        AND rso.snapshot = oas.snapshot
    JOIN {{.operatorShareSnapshotsTable}} oss
        ON rso.operator = oss.operator
        AND rso.strategy = oss.strategy
        AND rso.snapshot = oss.snapshot
)
```

In `17_goldStakerOperatorSetUniqueStakeRewards.go`, staker weights are computed from unscaled shares and then normalised per operator, without applying `magnitude / max_magnitude`:

```
-- Step 3: Get each staker's weighted shares across ALL strategies in the operator set
staker_strategy_shares AS (
    SELECT
        sd.reward_hash,
        sd.snapshot,
        sd.operator,
        sd.staker,
        SUM(sss.shares * asr.multiplier) as weighted_shares
    FROM staker_delegations sd
    JOIN {{.activeStakeRewardsTable}} asr
        ON sd.reward_hash = asr.reward_hash
        AND sd.avs = asr.avs
        AND sd.operator_set_id = asr.operator_set_id
        AND sd.snapshot = asr.snapshot
    JOIN staker_share_snapshots sss
        ON sd.staker = sss.staker
        AND asr.strategy = sss.strategy
        AND sd.snapshot = sss.snapshot
    GROUP BY sd.reward_hash, sd.snapshot, sd.operator, sd.staker
),
staker_weight_with_totals AS (
    SELECT
        *,
        SUM(staker_weight) OVER (PARTITION BY reward_hash, operator, snapshot) as total_operator_weight
    FROM staker_weights
)
```

As a result, stakers concentrated in under-allocated strategies can receive a larger share of the operator's staker split than intended, while stakers concentrated in heavily allocated strategies can be diluted.

Recommendations

Apply allocation scaling in Table 17 by joining `operator_allocation_snapshots` and computing staker weighted shares as `sss.shares * (oas.magnitude / oas.max_magnitude) * asr.multiplier`.

Add a regression test with multi-strategy operator sets and unequal allocation ratios to confirm staker splits follow allocation-adjusted stake composition.

Resolution

The issue was fixed by adding allocation scaling to staker weights in Table 17.

The issue has been addressed in the PR #500.

ELSD-02	Silent Failure In Staker Delegation Snapshot Generation		
Assets	`sidecar/pkg/rewards/stakerDelegationSnapshots.go`		
Status	**Resolved:** See Resolution		
Rating	Severity: Medium	Impact: High	Likelihood: Low

Description

The function `GenerateAndInsertStakerDelegationSnapshots()` contains a silent error handling bug. If `rewardsUtils.RenderQueryTemplate()` fails (e.g. due to template syntax errors or missing variable keys), the function logs the error but returns `nil` instead of the error object.

The calling function, `generateSnapshotData()` in `rewards.go`, interprets the `nil` return as a success. It proceeds with the rewards calculation pipeline without having executed the staker delegation snapshot query. This results in the `staker_delegation_snapshots` table being empty or stale for the given snapshot date. Consequently, subsequent calculations will fail to attribute stake to operators, leading to incorrect reward calculations.

```go
func (r *RewardsCalculator) GenerateAndInsertStakerDelegationSnapshots(snapshotDate string) error {
    query, err := rewardsUtils.RenderQueryTemplate(stakerDelegationSnapshotsQuery, map[string]interface{}{
        "cutoffDate": snapshotDate,
    })
    if err != nil {
        r.logger.Sugar().Errorw("Failed to render operator share snapshots query", "error", err)
        return nil // @audit Returns nil on error, suppressing the failure
    }

    res := r.grm.Exec(query)
    // ...
}
```

Recommendations

Modify the error handling block to return the error to the caller. This ensures the pipeline halts appropriately upon failure, allowing the system to retry or alert operators to the issue.

Resolution

The issue was fixed by changing the error handling to return the error instead of `nil`.

The issue has been addressed in the PR #500.

ELSD-03	Rewards-By-AVS Pagination Offset Uses Page Number Directly		
Assets	sidecar/pkg/service/rewardsDataService/rewards.go sidecar/pkg/rpcServer/rewardsHandlers.go		
Status	**Resolved:** See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Medium

Description

The pagination flow for `GetRewardsByAvsForDistributionRoot()` treats `Page` as a page number, but the SQL query uses it directly as row offset. This causes substantial page overlap and duplicate results when clients paginate by incrementing page number.

Handler flow confirms `Page` semantics as page number:

```
page = serviceTypes.NewDefaultPagination()
page.Load(pagination.PageNumber, pagination.PageSize)
...
nextPage = &common.Pagination{
    PageNumber: page.Page + 1,
    PageSize:   page.PageSize,
}
```

In the data service, offset is currently bound as `pagination.Page`:

```
order by gt.snapshot, cr.avs, gt.earner, gt.token desc
limit @limit
offset @offset
...
queryArgs = append(queryArgs, sql.Named("limit", pagination.PageSize))
queryArgs = append(queryArgs, sql.Named("offset", pagination.Page))
```

Example: with `pageSize = 100`, page 0 uses offset 0, but page 1 uses offset 1 (not 100), so 99 entries overlap with page 0. This behaviour is inconsistent with other paginated service queries that compute offset as `pagination.Page * pagination.PageSize`.

Recommendations

Compute offset as `pagination.Page * pagination.PageSize` in `GetRewardsByAvsForDistributionRoot()`. Add a regression test covering page 0 and page 1 with a fixed `pageSize` and assert no overlap between result sets.

Resolution

The issue has been addressed in the PR #500.

ELSD-04	Owner Split Misconfiguration Can Create Negative Staker Reward Math		
Assets	eigenlayer-contracts/src/contracts/core/RewardsCoordinator.sol sidecar/pkg/rewards/16_goldOperatorOperatorSetUniqueStakeRewards.go sidecar/pkg/rewards/17_goldStakerOperatorSetUniqueStakeRewards.go sidecar/pkg/rewards/19_goldOperatorOperatorSetTotalStakeRewards.go sidecar/pkg/rewards/20_goldStakerOperatorSetTotalStakeRewards.go		
Status	**Resolved:** See Resolution		
Rating	Severity: Low	Impact: Medium	Likelihood: Low

Description

This issue is triggered by trusted-owner misconfiguration. The default split path is owner-only but currently unbounded, while operator-specific split setters enforce the 10000 bips cap:

```
-- RewardsCoordinator.sol
function setDefaultOperatorSplit(uint16 split) external onlyOwner {
    _setDefaultOperatorSplit(split);
}

function _setDefaultOperatorSplit(uint16 split) internal {
    defaultOperatorSplitBips = split;
}

function _setOperatorSplit(...) internal {
    require(split <= ONE_HUNDRED_IN_BIPS, SplitExceedsMax());
    ...
}
```

Split configuration is separate from rewards submission. Unique/total stake submissions do not carry split values. If `setOperatorSetSplit()` is not configured, sidecar falls back to the default split snapshot (and finally to literal `1000` if no split snapshot exists):

```
-- RewardsCoordinator.sol
createUniqueStakeRewardsSubmission(OperatorSet operatorSet, RewardsSubmission[] rewardsSubmissions)
createTotalStakeRewardsSubmission(OperatorSet operatorSet, RewardsSubmission[] rewardsSubmissions)
setOperatorSetSplit(address operator, OperatorSet operatorSet, uint16 split)
```

Sidecar applies split using a fallback and does not clamp out-of-range values:

```
-- 16_goldOperatorOperatorSetUniqueStakeRewards.go (Step 5)
COALESCE(oss.split, dos.split, 1000) / CAST(10000 AS NUMERIC) AS split_pct,
FLOOR(dop.tokens_per_registered_snapshot_decimal * COALESCE(oss.split, dos.split, 1000) / CAST(10000 AS NUMERIC)) AS
      ↪  operator_tokens

-- 19_goldOperatorOperatorSetTotalStakeRewards.go (Step 5)
COALESCE(oss.split, dos.split, 1000) / CAST(10000 AS NUMERIC) AS split_pct,
FLOOR(dop.tokens_per_registered_snapshot_decimal * COALESCE(oss.split, dos.split, 1000) / CAST(10000 AS NUMERIC)) AS
      ↪  operator_tokens
```

Tables 17 and 20 compute staker pool as `base - operator_tokens`. If the effective split exceeds 10000, `staker_split_total` becomes negative, and delegated-staker rows compute negative payouts:

```
-- 17_goldStakerOperatorSetUniqueStakeRewards.go
adjusted_tokens_per_snapshot - operator_tokens as staker_split_total
...
FLOOR(staker_proportion * staker_split_total) as staker_tokens

-- 20_goldStakerOperatorSetTotalStakeRewards.go
tokens_per_registered_snapshot_decimal - operator_tokens as staker_split_total
...
FLOOR(staker_proportion * staker_split_total) as staker_tokens
```

Snapshots ingest split values as plain integers without sidecar-side range checks:

```
-- defaultOperatorSplitSnapshots.go
select dos.new_default_operator_split_bips as split

-- operatorSetSplitSnapshots.go
select oss.new_operator_set_split_bips as split

-- migrations
split integer not null
```

These values then flow through staging and final tables:

```
GenerateAndInsertDefaultOperatorSplitSnapshots(...)
GenerateAndInsertOperatorSetSplitSnapshots(...)
...
GenerateGold16... / GenerateGold17... / GenerateGold19... / GenerateGold20...
GenerateGold22StagingTable(...)
GenerateGold23FinalTable(...)
```

Example: if the admin/owner sets the default operator split to `20000` bips and no operator-set override is active, tables 16/19 allocate 200% to the operator, and tables 17/20 compute negative staker payouts for operators with non-zero delegated staker weight. Those values flow unchanged into tables 22/23.

Recommendations

Enforce `split <= ONE_HUNDRED_IN_BIPS` in `_setDefaultOperatorSplit()` so both `setDefaultOperatorSplit()` and `initialize()` share the same bound invariant as `_setOperatorSplit()`. Add fail-closed validation in sidecar split arithmetic and snapshot storage so out-of-range split values are rejected before reward math.

Resolution

Sidecar was updated to use `LEAST` of `dos.new_default_operator_split_bips` or `10000`

The issue has been addressed in the PR #500.

ELSD-05	Missing Strategy Registration Enforcement In V2.2 Stake Rewards Path		

Assets	`sidecar/pkg/rewards/stakeOperatorSetRewards.go`
	`sidecar/pkg/rewards/16_goldOperatorOperatorSetUniqueStakeRewards.go`
	`sidecar/pkg/rewards/19_goldOperatorOperatorSetTotalStakeRewards.go`		

Status	**Closed:** See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

The v2.2 stake rewards flow does not enforce operator-set strategy registration, so a rewards submission that references a strategy absent from `operator_set_strategy_registration_snapshots` can still be included in operator and staker distribution. This is a correctness issue if the intended policy is strict operator-set strategy membership; onchain validation for this path checks strategy whitelisting, not operator-set registration.

Call flow:

1. Stake submissions are copied into `stake_operator_set_rewards` as-is, with no validation against `operator_set_strategy_registration_snapshots`.
2. Table 15 (`active unique/total stake rewards`) explodes these rows by day and preserves the submitted strategy.
3. Tables 16 and 19 compute operator weights by joining only `operator_set_operator_registration_snapshots`, with no join to `operator_set_strategy_registration_snapshots`.

Ingestion keeps submitted strategies unchanged:

```
INSERT INTO stake_operator_set_rewards (..., strategy, ..., reward_type, ...)
SELECT
    usrs.avs,
    usrs.operator_set_id,
    usrs.reward_hash,
    usrs.token,
    usrs.amount,
    usrs.strategy,
    ...
FROM unique_stake_reward_submissions AS usrs
```

Table 15 uses those rows directly:

```
active_rewards_modified AS (
    SELECT
        avs,
        operator_set_id,
        reward_hash,
        token,
        amount,
        strategy,
        ...
    FROM stake_operator_set_rewards
)
```

Tables 16 and 19 gate by operator registration only:

```
WITH registered_operators AS (
    SELECT
        ap.reward_hash,
        ap.snapshot,
        ap.avs,
        ap.operator_set_id,
        ap.strategy,
        osor.operator
    FROM {{.activeStakeRewardsTable}} ap
    JOIN operator_set_operator_registration_snapshots osor
        ON ap.avs = osor.avs
       AND ap.operator_set_id = osor.operator_set_id
       AND ap.snapshot = osor.snapshot
    WHERE ap.reward_type = 'unique_stake' -- table 16
    -- WHERE ap.reward_type = 'total_stake' -- table 19
)
```

For comparison, the v2.1 operator-directed operator-set staker flow does enforce strategy registration:

```
JOIN operator_set_strategy_registration_snapshots ossr
    ON rso.avs = ossr.avs
   AND rso.operator_set_id = ossr.operator_set_id
   AND rso.snapshot = ossr.snapshot
   AND rso.strategy = ossr.strategy
```

As a result, if such strategies have nonzero shares (and, for unique stake, positive allocation), they can influence weighting in tables 16/19 and then flow into tables 17/20 (staker splits), creating a distribution that does not match operator-set strategy membership for that snapshot.

Recommendations

If strict strategy-registration semantics are desired for v2.2 stake rewards, enforce registration by joining `operator_set_strategy_registration_snapshots` in tables 16 and 19 (or equivalently pre-filtering in table 15). Ensure rows dropped by this validation are refunded to AVS or otherwise excluded from distribution accounting so invalid strategies cannot absorb rewards. Add regression tests for both unique-stake and total-stake submissions that include an unregistered strategy. If the current flexible behaviour is intended, document that policy explicitly in the rewards spec and implementation notes.

Resolution

The issue was acknowledged by the project team with the following comment:

> *"We discussed further and decided if a rewards submission includes a strategy, that strategy should be included in the weighting function for the duration of the reward regardless of whether it was registered. This gives an AVS the maximum flexibility to construct rewards functions how they intend without running into weird registration rules blocking them."*

ELSD-06	Token Filter SQL Uses Inconsistent Placeholders
Assets	sidecar/pkg/service/rewardsDataService/rewards.go sidecar/pkg/rpcServer/rewardsHandlers.go
Status	**Resolved:** See Resolution
Rating	Informational

Description

Token filtering in rewards data queries uses mixed placeholder styles inside single statements. Several queries append `IN (?)` while passing `sql.Named("tokens", ...)` and otherwise using `@named` placeholders. This mixture can mis-bind arguments for the `?` placeholder, yielding malformed or incorrect token filtering.

Examples from `rewards.go`:

```
if len(tokens) > 0 {
    query += " and token in (?)"      // also appears as "and gt.token in (?)"
    tokens = lowercaseTokenList(tokens)
    args = append(args, sql.Named("tokens", tokens))
}
```

`GetClaimableRewardsForEarner()` also appends token filtering after the JOIN block. That keeps the condition on the JOIN clause rather than applying it as a result-row filter:

```
select
    er.token,
    (coalesce(er.amount, 0) - coalesce(ct.amount, 0))::numeric as amount
from earner_rewards as er
left join claimed_tokens as ct on (
    ct.token = er.token
    and ct.earner = er.earner
)

if len(tokens) > 0 {
    query += " and token in (?)"
}
```

Current public API reachability is narrow:

```
// rewardsHandlers.go
GetClaimableRewards() -> GetClaimableRewardsForEarner(..., nil, ...)
GetTotalClaimedRewards() -> GetTotalClaimedRewards(..., nil, ...)
ListClaimedRewardsByBlockRange() -> ListClaimedRewardsByBlockRange(..., nil)
GetSummarizedRewardsForEarner() -> GetSummarizedRewards(..., nil, ...)
ListEarnerLifetimeRewards() -> GetTotalRewardsForEarner(..., nil, ...)

ListEarnerHistoricalRewards() -> ListHistoricalRewardsForEarner(..., tokens)

Only \code{ListEarnerHistoricalRewards()} currently forwards caller-provided token filters. The
other listed public endpoints pass \code{nil} for tokens.
```

In the public API schema, `GetClaimableRewardsRequest()`, `GetTotalClaimedRewardsRequest()`, `GetSummarizedRewardsForEarnerRequest()`, and `ListClaimedRewardsByBlockRangeRequest()` do not expose token filters. `ListEarnerHistoricalRewardsRequest()` is the only one that includes `tokens`.

Recommendations

Use one parameter style per query for token filters. Here, prefer named parameters consistently, with qualified columns (for example, `rc.token in @tokens`, `gt.token in @tokens`, and `er.token in @tokens`).

In `GetClaimableRewardsForEarner()`, move token filtering into an explicit `WHERE` clause on the selected reward rows (for example `er.token`) rather than appending it to the JOIN condition.

Resolution

The issue has been addressed in the PR #500.

Appendix A Vulnerability Severity Classification

This security review classifies vulnerabilities based on their potential impact and likelihood of occurance. The total severity of a vulnerability is derived from these two metrics based on the following matrix.

Impact	Likelihood: Low	Likelihood: Medium	Likelihood: High
High	Medium	High	Critical
Medium	Low	Medium	High
Low	Low	Low	Medium

Table 1: Severity Matrix - How the severity of a vulnerability is given based on the *impact* and the *likelihood* of a vulnerability.

σ'